

Capítulo 21: Diseño de Estructuras y Algoritmos Avanzados

1. Explicación Teórica: Inmutabilidad, Algoritmos Funcionales y Complejidad

El diseño avanzado de estructuras y algoritmos en Scala se basa en tres pilares interconectados que definen la Programación Funcional Pura (PFP): la **inmutabilidad**, la **composición declarativa** y la **eficiencia algorítmica**.

Estructuras de Datos Persistentes e Inmutables

En Scala, las colecciones son, por defecto, **inmutables**. Esto significa que cualquier "modificación" de una estructura existente (como añadir un elemento a una lista) no altera la original, sino que **devuelve una nueva colección**. Esta práctica es crucial para crear **sistemas concurrentes seguros** al eliminar el riesgo de condiciones de carrera (*race conditions*) asociadas al estado mutable compartido.

Cuando se diseña software avanzado, es fundamental elegir la estructura de datos correcta basándose en el análisis de rendimiento:

- **Listas (`List`)**: Son la colección fundamental inmutable y están optimizadas para la **recursión** y las operaciones de anteposición (agregar al inicio o `cons`).
- **Vectores (`Vector`)**: Se prefieren sobre `List` en escenarios donde se necesita acceso por índice con una complejidad garantizada de $O(1)$.
- **Estructuras Personalizadas (*Homegrown Collections*)**: Para lograr la verdadera maestría, el desarrollador debe ser capaz de implementar estructuras complejas (como árboles binarios de búsqueda inmutables o conjuntos) utilizando los principios funcionales (`case class`, `sealed trait`).

Algoritmos Funcionales y Complejidad ($O(n)$)

En lugar de construir algoritmos utilizando iteración imperativa (`for` o `while` con variables mutables `var`), Scala emplea dos técnicas principales para el diseño de algoritmos a gran escala:

1. **Funciones de Orden Superior (HOFs)**: Métodos como `map`, `filter`, `fold` y `reduce` permiten definir la lógica **declarativamente** (el *qué* hacer) en lugar de imperativamente (el *cómo* iterar).
2. **Recursión**: Es el mecanismo preferido para reemplazar los bucles iterativos. En un contexto funcional, el algoritmo llama a la función sobre un subconjunto de los datos hasta alcanzar un caso base.

El análisis teórico de la **complejidad** $O(n)$ (notación de orden) es esencial para determinar la eficiencia de estas estructuras y algoritmos, asegurando que las operaciones clave (como inserción, búsqueda o eliminación) cumplan con los requisitos de rendimiento esperados (por ejemplo, $O(1)$ para las operaciones de pilas y colas).

2. Sintaxis y Estructura: Modelado Inmutable y Recursión

A. Tipos de Datos Algebraicos (ADTs) para Estructuras

Las estructuras de datos inmutables se modelan utilizando **ADTs (Tipos de Datos Algebraicos)**, combinando `sealed trait` (o `enum` en Scala 3) con `case class`.

```
1 // El ADT base para una Lista Simple (similar a List.scala)
2 sealed trait MyList[+T]
3
4 // Tipo de Producto 1: La lista vacía (Singleton Object)
5 case object MyNil extends MyList[Nothing]
6
7 // Tipo de Producto 2: La lista no vacía (Case Class Cons)
8 final case class MyCons[+T](
9     head: T,           // Primer elemento (immutable)
10    tail: MyList[T]     // Resto de la lista (immutable)
11 ) extends MyList[T]
```

B. Sintaxis de Reducción Funcional (`reduce`)

Para aplicar un algoritmo de **reducción** (que resume toda una colección en un único valor), se utiliza `reduce` o `fold`.

```
1 // Definición genérica de una operación de reducción (requiere un tipo A)
2 def sumar(list: MyList[Int]): Int =
3     // Map/Reduce se aplican internamente usando Pattern Matching y
4     // recursión.
5     list match {
6         // Caso base: La lista vacía devuelve 0 (usando foldLeft con un valor
7         // inicial de 0).
8         case MyNil => 0
9         // Caso recursivo: Suma la cabeza y llama recursivamente a la suma
10        // sobre la cola.
11        case MyCons(h, t) => h + sumar(t)
12        // Nota: El compilador exige la declaración del tipo de retorno (Int)
13        // para la recursión.
14    }
```

3. Ejemplos de Código: Reducción (Algoritmo `reduce`)

El algoritmo `reduce` (`foldLeft` / `foldRight`) es la forma canónica de aplicar una lógica sobre toda la colección para obtener un valor final. Aquí simulamos la implementación de `reduceLeft` para encontrar el elemento máximo en una lista (algoritmo de búsqueda lineal, $O(n)$).

Caso de Uso Realista: Encontrar el jugador con la puntuación más alta en una lista de resultados de un juego.

```
1  case class Jugador(nombre: String, score: Int)
2
3  // Definimos la lista inmutable usando la estructura MyCons/MyNil (del
4  punto 2)
5  val resultados = MyCons(
6    Jugador("Max", 5000),
7    MyCons(Jugador("Ana", 8000),
8      MyCons(Jugador("Leo", 2500), MyNil)
9    )
10 )
11 // Función genérica recursiva que simula reduce/fold para encontrar el
12 // máximo.
13 // Requiere que el tipo T sea comparable (usando un Type Class implícito:
14 // Ordering[T])
15 def encontrarMaximo(list: MyList[Jugador]): Option[Jugador] = {
16   def foldRec(actual: Jugador, tail: MyList[Jugador]): Jugador =
17     tail match {
18       case MyNil => actual // Caso base: si la cola está vacía, devuelve
19       el 'actual'.
20       // Caso recursivo: Compara el 'actual' con la 'cabeza' de la cola.
21       case MyCons(h, t) =>
22         val siguienteMax = if (h.score > actual.score) h else actual
23         foldRec(siguienteMax, t) // Llama recursivamente con el nuevo
24         máximo.
25     }
26   list match {
27     case MyNil => None // Si la lista está vacía, no hay máximo.
28     case MyCons(h, t) => Some(foldRec(h, t))
29   }
30 }
31 // Ejecución y resultado (usando Pattern Matching para desestructurar el
32 // Option):
```

```

32  val jugadorGanador = encontrarMaximo(resultados) match {
33      case Some(j) => s"Ganador: ${j.nombre} con ${j.score}"
34      case None    => "No hay resultados."
35  }
36
37  // Resultado esperado: Ganador: Ana con 8000

```

Este ejemplo usa **Pattern Matching** para desestructurar las `case class` (`MyCons` y `MyNil`), y la **recursión** para iterar sobre la estructura inmutable, un patrón algorítmico fundamental en PF.

4. Comparativa con Java y Python

La principal diferencia en el diseño de estructuras y algoritmos en Scala radica en su adherencia al paradigma funcional y el sistema de tipos estático:

Aspecto	Java (Imperativo/Tradicional)	Python (Dinámico/Comprensiones)	Scala (Funcional)
Estructuras de Datos	Predominan estructuras mutables (<code>ArrayList</code> , <code>Array</code>).	Estructuras dinámicas (listas) mutables por defecto.	Inmutabilidad (<code>Vector</code>),.. "modificación siempre crea nueva estructura"
Algoritmos/Iteración	Uso de bucles <code>for / while</code> y mutación en su lugar (<i>in-place</i>).	Bucles, o <i>list comprehensions</i> para inmutabilidad simple.	HOFs (<code>map</code>), recursión mutación y intermedio
Seguridad de Tipos	El <i>casting</i> y la reflexión son necesarios para estructuras complejas.	Tipado dinámico; los errores de estructura se detectan en <i>runtime</i> .	Seguridad total . El <code>Pattern Matching</code> desestructura complejos <code>case class</code> en compilación
Notación O(n)	Usualmente se asume; la complejidad puede verse comprometida por la mutabilidad y la sincronización.	N/A.	Se espera diseño de estructuras de datos inmutables y persistentes y garanticen complejidad algorítmica

5. Best Practices (*The Scala Way*)

1. **Inmutabilidad Rigurosa:** El diseño de cualquier estructura de datos debe comenzar con `val` y `case class` . La mutabilidad (`var`) solo debe usarse en contextos de micro-optimización de bajo nivel o dentro de una unidad encapsulada (como un Actor).
2. **HOFs para Algoritmos:** Evita escribir bucles `for` o `while` al estilo imperativo, ya que requieren `var` . Para cualquier algoritmo de transformación o reducción, utiliza los métodos `map` , `filter` , `flatMap` y `fold` . El uso de `for-comprehensions` es azúcar sintáctico para estas HOFs y mejora la legibilidad en cadenas complejas.
3. **Recursión vs. Iteración:** Utiliza la **recursión** como la técnica fundamental para la iteración en lógica funcional. Para evitar errores de desbordamiento de pila (*Stack Overflow*), la recursión debe ser **recursión de cola optimizada (TCO)**, donde la llamada recursiva es la última operación en la función.
4. **Modelado con ADTs:** Al crear estructuras de datos complejas (árboles, grafos, listas anidadas), utiliza el binomio `sealed trait/enum` y `case class` para modelar sus variantes. Esto permite usar el **Pattern Matching** para desestructurar la lógica de los algoritmos de forma concisa y segura en tipos,.
5. **Eficiencia de Colecciones:** Aunque `List` es la opción por defecto, usa `Vector` si necesitas un acceso rápido (cercano a $O(1)$) a elementos por índice, ya que su rendimiento es superior al de `List` para esta operación.

El diseño avanzado de algoritmos en Scala no es solo una cuestión de velocidad; es una cuestión de **verificación y composicionalidad**. Al utilizar estructuras inmutables y composición funcional, el desarrollador construye un algoritmo como si estuviera encadenando operaciones matemáticas (HOFs), donde la seguridad y la corrección del tipo son probadas por el compilador, mientras que la eficiencia algorítmica ($O(n)$) se mantiene gracias a las propiedades de la estructura subyacente.